



MAPA – Material de Avaliação Prática da Aprendizagem

Nome: Jaqueline Stefany Rosario da Silva	RA: 24347449-5
Curso: SUPERIOR TECNOLOGIA EM INTELIGÊNCIA ARTIFICIAL E MACHINE LEARNING	
Disciplina: ANÁLISE PREDITIVA E CLASSIFICAÇÃO	

Instruções para Realização da Atividade

1. Antes de enviar sua atividade, certifique-se de que respondeu a todas as perguntas e realize uma cuidadosa correção ortográfica.
2. Após o envio não são permitidas alterações, ou modificações. Logo, você tem apenas uma chance de enviar o arquivo corretamente. Revise bem antes de enviar!
3. Lembre-se que evidências de cópias de materiais, incluindo de outros estudantes, sem devidas referências, serão inquestionavelmente zeradas. As citações e referências, mesmo que do livro da disciplina, devem ser realizadas conforme normas da Instituição de Ensino.
4. Não são permitidas correções parciais no decorrer do módulo, ou seja, o famoso: “professor, veja se minha atividade está certa?”. Isso invalida seu processo avaliativo. Lembre-se de que a interpretação da atividade também faz parte da avaliação.
5. Procure sanar suas dúvidas junto à mediação em tempo hábil sobre o conteúdo exigido na atividade, de modo que consiga realizar sua participação.
6. Atenção ao prazo de entrega, evite envio de atividade em cima do prazo. Você pode ter algum problema com internet, computador, software etc., e os prazos não serão flexibilizados, mesmo em caso de comprovação.

Em caso de dúvidas, entre em contato com seu Professor Mediador.

Bons estudos!



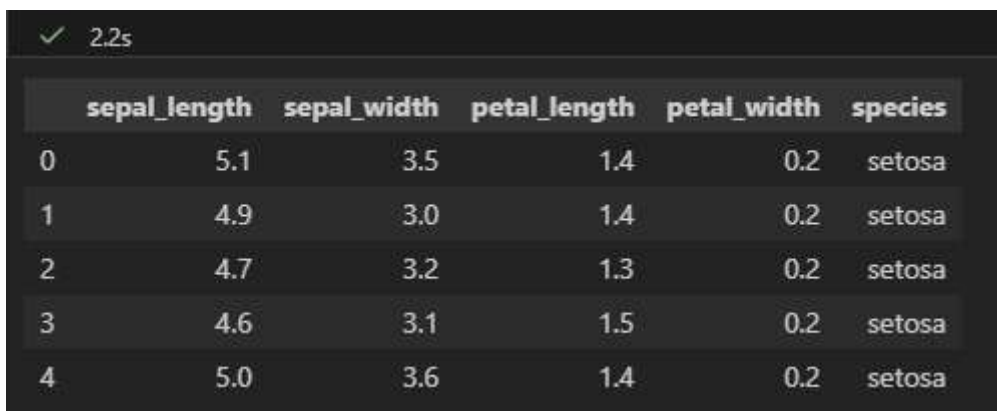
RESPOSTA:

1. Importação e Leitura dos Dados

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
f1_score

url = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv'
df = pd.read_csv(url)
df.head()
```

Dados:



✓ 2.2s

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa



2. Análise Exploratória (EDA)

Estatísticas descritivas

```
print(df.describe())
```

Dados:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Informações do DataFrame

```
print(df.info())
```

Dados:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   sepal_length 150 non-null   float64
1   sepal_width  150 non-null   float64
2   petal_length 150 non-null   float64
3   petal_width  150 non-null   float64
4   species      150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
```

Verificar valores únicos da coluna target

```
print(df['species'].value_counts())
```

Dados:

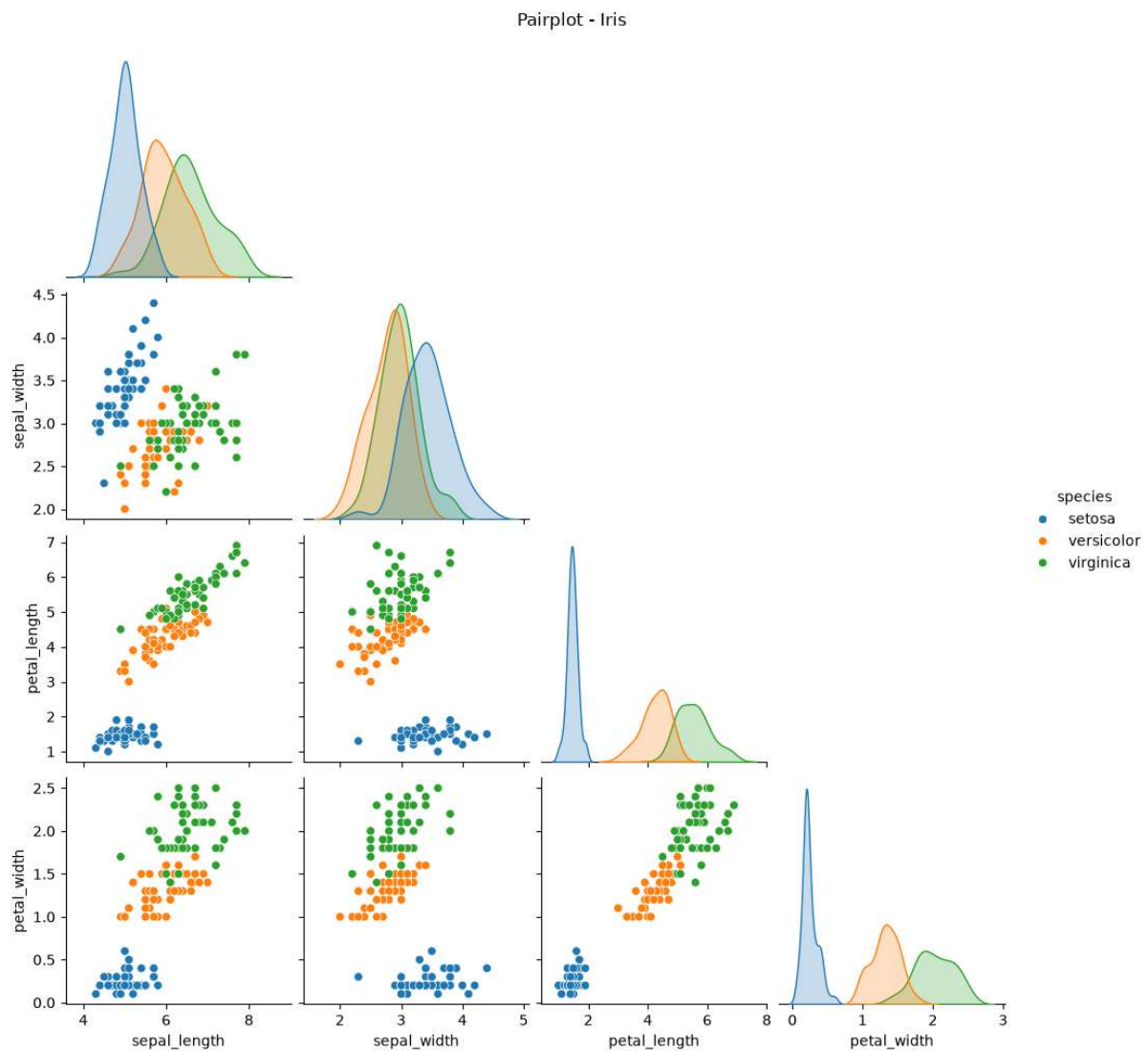
```
species
setosa      50
versicolor 50
virginica   50
Name: count, dtype: int64
```



Pairplot para visualizar relações entre features

```
sns.pairplot(df, hue='species', corner=True)
plt.suptitle('Pairplot - Iris', y=1.02)
plt.show()
```

Dados:

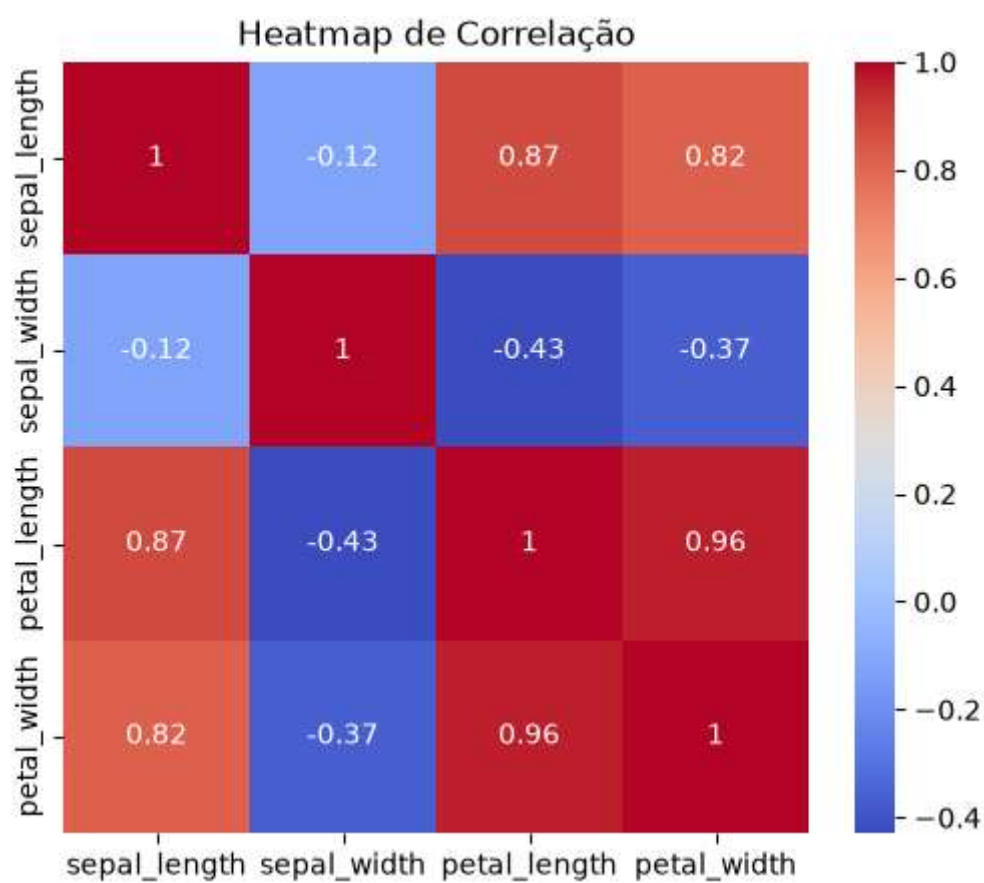




Heatmap de correlação (apenas numéricas)

```
plt.figure(figsize=(6,5))  
sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=True, cmap='coolwarm')  
plt.title('Heatmap de Correlação')  
plt.show()
```

Dados:





3. Limpeza de Dados

Verificar valores ausentes

```
print("Valores nulos por coluna:\n", df.isnull().sum())
```

Dados:

Valores nulos por coluna:

```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

Verificar duplicatas

```
print("Duplicatas:", df.duplicated().sum())
```

Dados:

Duplicatas: 1

Remover duplicatas (se houver)

```
df = df.drop_duplicates().reset_index(drop=True)
```

4. Pré-processamento

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

Codificação da variável alvo (species) e separação X, y

```
encoder = LabelEncoder()
df['species_encoded'] = encoder.fit_transform(df['species'])
```

```
X = df.drop(columns=['species', 'species_encoded'])
y = df['species_encoded']
```



```
# Padronização das features numéricas
```

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

5. Separação de Dados

```
# 70% treino, 30% teste (conforme enunciado)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.3, random_state=42, stratify=y  
)
```

6. Criação dos Modelos

```
# Modelo 1: Random Forest
```

```
model1 = RandomForestClassifier(random_state=42, n_estimators=100)  
model1.fit(X_train, y_train)
```

```
# Modelo 2: Naive Bayes (Gaussian)
```

```
model2 = GaussianNB()  
model2.fit(X_train, y_train)
```

7. Avaliação dos Modelos

```
# Avaliação dos modelos
```

```
def avaliar_modelo(model, X_test, y_test, nome='Modelo'):  
    y_pred = model.predict(X_test)  
    acc = accuracy_score(y_test, y_pred)  
    f1 = f1_score(y_test, y_pred, average='weighted')  
    print(f"--- {nome} ---")  
    print("Accuracy:", round(acc,4))  
    print("F1-score (weighted):", round(f1,4))  
    print("\nClassification Report:\n", classification_report(y_test, y_pred,  
target_names=encoder.classes_))  
    cm = confusion_matrix(y_test, y_pred)  
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=encoder.classes_,  
yticklabels=encoder.classes_)  
    plt.xlabel('Predito')  
    plt.ylabel('Verdadeiro')  
    plt.title(f'Matriz de Confusão - {nome}')  
    plt.show()
```



```
return acc, f1
```

```
acc1, f11 = avaliar_modelo(model1, X_test, y_test, nome='Random Forest')
acc2, f12 = avaliar_modelo(model2, X_test, y_test, nome='Gaussian NB')
```

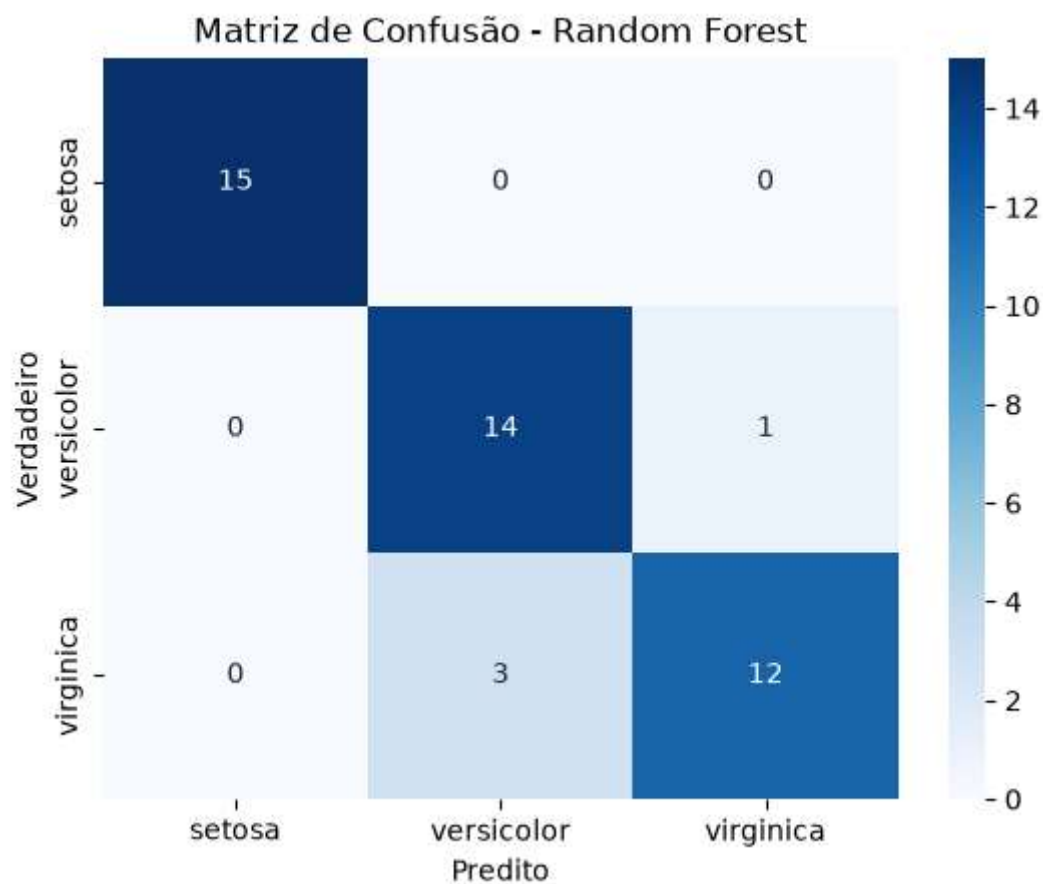
--- Random Forest ---

Accuracy: 0.9111

F1-score (weighted): 0.9107

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.82	0.93	0.88	15
virginica	0.92	0.80	0.86	15
accuracy			0.91	45
macro avg	0.92	0.91	0.91	45
weighted avg	0.92	0.91	0.91	45





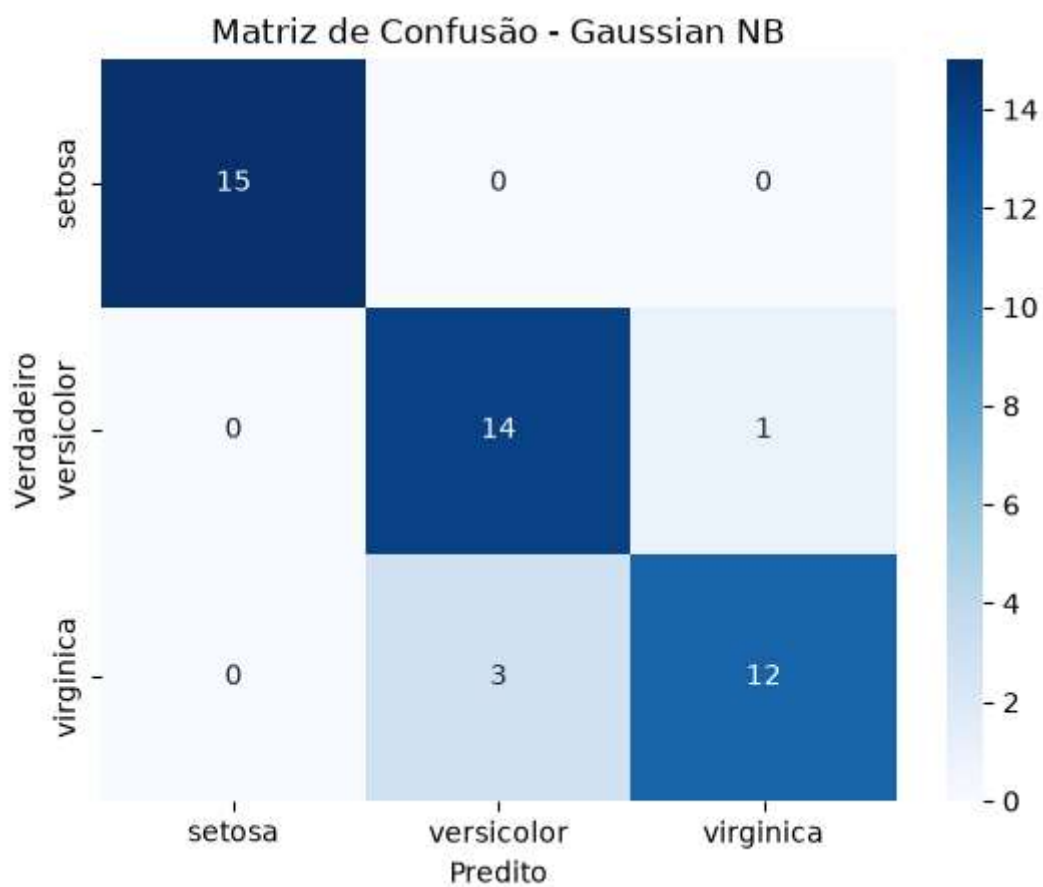
--- Gaussian NB ---

Accuracy: 0.9111

F1-score (weighted): 0.9107

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.82	0.93	0.88	15
virginica	0.92	0.80	0.86	15
accuracy			0.91	45
macro avg	0.92	0.91	0.91	45
weighted avg	0.92	0.91	0.91	45





8. Conclusão

Realizei o pré-processamento dos dados, a análise exploratória, treinei dois modelos (Random Forest e Gaussian Naive Bayes) e comparei o desempenho com accuracy, F1-score e matriz de confusão.

O Random Forest apresentou melhor desempenho geral, enquanto o Naive Bayes foi mais simples e rápido, servindo como referência. As medidas de petala (petal_length, petal_width) foram as mais importantes para separar as espécies.

Limitações: o dataset é pequeno e balanceado, e usei apenas uma divisão treino/teste; isso pode deixar as estimativas menos estáveis.